

Reviewer Pack — Jordan Rank Lower Bound for Tseitin Resolution Width

Entry 6c4faf4f8bc5 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 14:38:35 UTC

Jordan Rank Lower Bound for Tseitin Resolution Width

Entry ID: 6c4faf4f8bc5 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-19 14:38:35 UTC

1. Conjecture

Field A (mathematical branch): Jordan Algebras **Field B** (complexity object): Resolution Width

Statement:

For a Tseitin formula derived from a d -regular expander graph with n vertices, the minimal Jordan rank of its clause incidence matrix over $\text{GF}(2)$ is $\Omega(\sqrt{n})$. This implies that the resolution width of the formula is $\Omega(\sqrt{n})$ for all such instances.

Rationale (proposer’s reasoning):

Jordan algebras provide a framework to analyze symmetric structures in clause incidence matrices. The Jordan rank captures non-linear dependencies in the clauses, which may correlate with the inherent width required for resolution proofs in expander-based Tseitin formulas.

Taxonomy category: PROOF_COMPLEXITY_TSEITIN (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 4f463eb1200311a0

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	UNCERTAIN	SAFE
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(matrix):
    n = len(matrix)
    for i in range(n):
        # Find pivot row and swap with current row
        max_row = i
        for k in range(i+1, n):
            if abs(matrix[k][i]) > abs(matrix[max_row][i]):
                max_row = k
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]

        # Eliminate non-pivot elements below the pivot
        factor = Fraction(1, matrix[i][i])
        for k in range(i+1, n):
            matrix[k][i] *= -factor

        # Eliminate non-pivot elements above and below the current row
        for j in range(n):
            if j != i:
                factor = matrix[j][i]
                for k in range(n):
                    matrix[j][k] += factor * matrix[i][k]

    # Count non-zero rows to get Jordan rank
    jordan_rank = sum(1 for row in matrix if any(row))
    return jordan_rank

def d_regular_expander_graph(d, n):
    graph = [[] for _ in range(n)]
    for i in range(n):
        neighbors = random.sample(range(n), d)
        while len(neighbors) > 0:
            neighbor = neighbors.pop()
            if neighbor != i and i not in graph[neighbor]:
                graph[i].append(neighbor)
                graph[neighbor].append(i)
    return graph

def clause_incidence_matrix(graph, n):
    matrix = [[0] * n for _ in range(n)]
    for u in range(n):
        for v in graph[u]:
```

```

        if u < v:
            matrix[u][v] = 1
            matrix[v][u] = 1
    return matrix

def resolution_width(matrix, n):
    # Placeholder function; actual implementation needed
    return 0 # Replace with actual width estimation logic

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [5, 10, 15, 20, 30, 40]
    total_metric_value = 0
    instances_tested = 0

    for n in n_values:
        d = 3 # Example value; adjust as needed
        graph = d_regular_expander_graph(d, n)
        matrix = clause_incidence_matrix(graph, n)

        jordan_rank = gaussian_elimination(matrix)
        width = resolution_width(matrix, n)

        if jordan_rank >= math.sqrt(n):
            total_metric_value += width
            instances_tested += 1

    metric_name = "resolution_width"
    metric_value = total_metric_value / instances_tested if instances_tested > 0 else 0
    conjecture_holds = metric_value >= math.sqrt(n_values[-1])
    counterexample = "" if conjecture_holds else f"width < sqrt({n_values[-1]})"

    return {
        "metric_name": metric_name,
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19,

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
    std_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value) ** 2 for result
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):

```

```

        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5073fcf6.py", line 106, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5073fcf6.py", line 80, in run_trial
    jordan_rank = gaussian_elimination(matrix)
                  ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5073fcf6.py", line 29, in gaussian_elimination
    factor = Fraction(1, matrix[i][i])
              ^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/fractions.py", line 281, in __new__
    raise ZeroDivisionError('Fraction(%s, 0)' % numerator)
ZeroDivisionError: Fraction(1, 0)

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with ZeroDivisionError, preventing data collection. No counterexample found but support condition unmet. | next: Fix Gaussian elimination to handle singular matrices or verify matrix construction validity

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	112467
2	propose	ollama_remote	qwen3:8b	0	0	101405
3	preregistration	ollama_remote	qwen3:8b	0	0	27816
4	novelty	ollama_remote	qwen3:8b	0	0	24079
5	novelty	ollama_remote	qwen3:8b	0	0	19797

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15658
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13508
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12055
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11968
10	judge	ollama_remote	qwen3:8b	0	0	22994

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 361748 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in *research/audit/6c4faf4f8bc5.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/6c4faf4f8bc5.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/6c4faf4f8bc5.tar.gz* (if generated)